

VisaPath: Architecture & Technical Documentation

Table of Contents

- System Architecture
- Request Flows
- API Endpoints
- Timeline Generation Engine
- Risk Analysis Engine
- RAG Pipeline
- AI Chat Service
- Data Layer
- Authentication
- Rate Limiting
- Frontend Architecture

System Architecture

```
graph TB
    subgraph User["User (Browser)"]
        Browser["Web Browser"]
    end

    subgraph Frontend["Frontend: React 19 + Vite 7 + Tailwind v4 + React Router v7"]
        AUTH["Auth Screen<br/><i>Login / Register</i>"]
        OF["Onboarding Form"]
        TD["Timeline Dashboard"]
        WIF["What-If Simulator"]
        CP["AI Chat Panel"]
        DT["Document Tracker"]
        TXG["Tax Guide"]
        API_CLIENT["API Client"]
        AUTH & OF & TD & WIF & CP & DT & TXG --> API_CLIENT
    end

    subgraph Backend["Backend: Python 3.13 + FastAPI"]
        subgraph Routes["API Routes"]
            R_AUTH["Auth Routes<br/><i>register, login, me, profile</i>"]
            R1["POST /api/generate-timeline"]
            R2["POST /api/chat"]
            R3["GET /api/required-documents"]
            R4["POST /api/tax-guide"]
            R5["GET /api/rate-limit-status"]
        end
    end
```

```

subgraph Middleware["Middleware"]
    RL["Rate Limiter<br/><i>Per-IP + AI daily limit</i>"]
    JWT["JWT Auth<br/><i>Token validation</i>"]
end

subgraph Services["Services"]
    TG["AI Timeline Generator<br/><i>Gemini powered with fallback</i>"]
    RA["Risk Analyzer<br/><i>Flags CPT, backlog, deadline risks</i>"]
    CS["Chat Service"]
    RAG["RAG Service<br/><i>Embed > Search > Retrieve</i>"]
    GS["Gemini Service<br/><i>Prompt assembly + API call</i>"]
    AS["Auth Service<br/><i>bcrypt hashing + JWT</i>"]
end

subgraph Data["Data Layer"]
    DB["SQLite / PostgreSQL<br/><i>Users, profiles, cached results</i>"]
    IR["immigration_rules.py<br/><i>56 rules + filing fees</i>"]
    SC["stem_cip_codes.py"]
    CB["country_backlogs.py"]
    DR["document_requirements"]
end

subgraph VectorDB["Vector Store"]
    CHROMA["ChromaDB<br/><i>21+ embedded chunks</i>"]
end

R_AUTH --> AS --> DB
R1 --> TG & RA
R2 --> CS
R3 --> DR
R4 --> GS
TG & RA --> IR & SC & CB
TG --> GS
CS --> RAG --> CHROMA
CS --> GS

end

subgraph External["External APIs"]
    GEMINI["Google Gemini API<br/><b>gemini-2.5-flash</b> / <b>gemini-2.0-flash</b><br/>"]
end

Browser -->|HTTPS| Frontend
API_CLIENT -->|REST API / JSON| Routes
Routes --> Middleware
GS --> GEMINI
RAG -->|Embedding requests| GEMINI

```

Request Flows

Timeline Generation

```
sequenceDiagram
    actor User
    participant FE as Frontend
    participant API as POST /api/generate-timeline
    participant TG as AI Timeline Generator
    participant RA as Risk Analyzer
    participant Data as Data Layer (Rules + Fees + Backlogs)
    participant Gemini as Gemini 2.5 Flash

    User->>FE: Fills onboarding form
    FE->>FE: Navigate to /timeline immediately (fire and forget)
    FE->>API: POST with user input + user_today (browser local date)
    API->>TG: generate_timeline(user_input, user_today)
    TG->>Data: Read 56 USCIS rules, filing fees, H-1B lottery stats, country backlogs
    Data-->>TG: Rules, fees, backlogs, STEM CIP codes
    TG->>TG: Assemble detailed prompt with all rules + user context
    TG->>Gemini: Send structured prompt, request JSON timeline
    Gemini-->>TG: Structured JSON (events, risks, status)
    TG->>TG: Validate JSON schema, recalculate is_past from user_today
    TG-->>API: timeline_events[]
    API->>RA: analyze_risks(user_input, events)
    RA->>Data: Read country_backlogs, thresholds
    Data-->>RA: Backlog data, risk thresholds
    RA-->>API: risk_alerts[]
    API-->>FE: { timeline_events, risk_alerts, current_status }
    FE->>FE: Recalculate is_past + urgency from browser date (useMemo)
    FE-->>User: Render interactive timeline
```

AI Chat (RAG Pipeline)

```
sequenceDiagram
    actor User
    participant FE as Frontend
    participant API as POST /api/chat
    participant RAG as RAG Service
    participant Chroma as ChromaDB (21 chunks)
    participant Gemini as Gemini 2.5 Flash

    User->>FE: "Can I work for two employers on STEM OPT?"
    FE->>API: POST { message, user_context }
    API->>RAG: retrieve_context(query)
    RAG->>Gemini: Embed query (gemini-embedding-001, 768 dims)
    Gemini-->>RAG: Query vector
```

```

RAG->>Chroma: Cosine similarity search (top k=4)
Chroma-->>RAG: 4 most relevant document chunks
RAG-->>API: RAG context string with source metadata
API->>Gemini: System prompt + User context + RAG chunks + Question
Gemini-->>API: Grounded response with source references
API-->>FE: { response, has_sources }
FE-->>User: Display answer with citations

```

API Endpoints

POST /api/generate-timeline

Generates a personalized immigration timeline using AI.

Request Body:

```

{
  "visa_type": "F-1",
  "degree_level": "Master's",
  "is_stem": true,
  "program_start": "2024-08-20",
  "expected_graduation": "2026-05-15",
  "cpt_months_used": 6,
  "currently_employed": true,
  "has_job_offer": true,
  "employer_is_cap_exempt": false,
  "wage_level": 2,
  "opt_status": "not_yet",
  "opt_ead_end_date": "",
  "career_goal": "stay_us_longterm",
  "country": "India",
  "user_today": "2026-02-17"
}

```

Response:

```

{
  "timeline_events": [
    {
      "id": "opt_apply_window_open",
      "title": "OPT Application Window Opens",
      "date": "2026-02-14",
      "type": "deadline",
      "urgency": "critical",
      "description": "You can start applying for post-completion OPT...",
      "action_items": ["Request OPT recommendation from DSO", "..."],
      "is_past": false
    }
  ]
}

```

```

],
"risk_alerts": [
  {
    "type": "country_backlog",
    "severity": "warning",
    "message": "As a national of India, EB-2/EB-3 green card wait times...",
    "recommendation": "Consider EB-1 eligibility..."
  }
],
"current_status": {
  "visa": "F-1",
  "work_auth": "Student (CPT/On-Campus)",
  "days_until_next_deadline": 1,
  "next_deadline": "OPT Application Window Opens"
}
}

```

POST /api/chat

AI Q&A with RAG context retrieval.

Request Body:

```

{
  "message": "Can I work for two employers on STEM OPT?",
  "user_context": {
    "visa_type": "F-1",
    "degree_level": "Master's",
    "is_stem": true,
    "country": "India"
  }
}

```

Response:

```

{
  "response": "Yes, you can work for multiple employers on STEM OPT, but each employer must",
  "has_sources": true
}

```

POST /api/tax-guide

AI generated personalized tax filing guide.

Request Body:

```

{
  "visa_type": "F-1",
  "country": "India",

```

```

    "has_income": true,
    "income_types": ["wages"],
    "years_in_us": 2
  }

```

Response:

```

{
  "filing_deadline": "April 15, 2026",
  "residency_status": "Nonresident Alien",
  "required_forms": ["Form 8843", "Form 1040-NR"],
  "treaty_benefits": { "country": "India", "benefit": "...", "form": "Form 8233" },
  "fica_exempt": true,
  "guidance": "### Filing Requirements\n...",
  "disclaimer": "This is general guidance, not legal or tax advice."
}

```

GET /api/required-documents?step=opt_application

Returns document checklists for immigration steps.

Available steps: opt_application, stem_opt_extension, h1b_petition, green_card_perm

GET /api/rate-limit-status

Returns current AI usage so the frontend can pre-check before triggering expensive AI calls.

Response:

```

{
  "used": 5,
  "limit": 1500,
  "remaining": 1495,
  "allowed": true,
  "retry_after": 0
}

```

Auth Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Create account (email + password)
/api/auth/login	POST	Login, returns JWT token
/api/auth/me	GET	Get current user profile + cached data
/api/auth/profile	PUT	Save/update user profile
/api/auth/cached-timeline	PUT	Cache AI generated timeline
/api/auth/cached-tax-guide	PUT	Cache AI generated tax guide

Endpoint	Method	Description
/api/auth/save-timeline	POST	Save timeline to history
/api/auth/my-timelines	GET	Get saved timeline history

Timeline Generation Engine

File: backend/app/services/ai_timeline_generator.py

The AI timeline generator assembles a comprehensive prompt with all immigration context and sends it to Gemini 2.5 Flash for structured JSON output.

Prompt Assembly

The prompt includes: 1. **User profile:** visa type, degree, STEM status, graduation date, employment, career goal, country 2. **56 hardcoded USCIS rules:** OPT windows, STEM OPT extensions, H-1B lottery, cap-gap, unemployment limits 3. **USCIS filing fees:** \$580 I-765 (OPT), \$780 I-129 (H-1B), \$215 H-1B registration, \$2,805 premium processing 4. **H-1B lottery statistics:** FY2024 through FY2027 registration counts, selection rates, wage level weighted selection 5. **Country backlog data:** EB-1/EB-2/EB-3 wait times by country 6. **Today's date:** from user's browser (timezone correct)

Supported Visa Pathways

- F-1 > OPT > STEM OPT > H-1B > Green Card
- F-1 > H-1B (direct, cap exempt employers)
- OPT > STEM OPT > H-1B
- H-1B > Green Card (PERM > I-140 > I-485)

Urgency Calculation

The frontend recalculates urgency on every render so cached timelines never show stale data:

Urgency	Condition	Color
critical	7 days away or less	Red
high	30 days away or less	Amber
medium	90 days away or less	Teal
low	More than 90 days away	Slate
passed	Date is in the past	Dim

Risk Analysis Engine

File: backend/app/services/risk_analyzer.py

Evaluates user input and flags potential immigration issues.

Risk	Severity	Trigger
CPT overuse (12+ months)	Critical	Kills OPT eligibility
CPT approaching limit (9+ months)	Warning	Close to losing OPT
Country backlog (India/China)	Warning	10-30+ year green card waits
OPT deadline approaching (<30 days)	Critical	Risk of missing OPT window
Non-STEM limited OPT	Info	Only 12 months, no extension
Unemployment on OPT	High	90/150 day limits
H-1B lottery uncertainty	Info	~25-30% selection rate
Low wage level + H-1B FY2027+	Warning	Lower selection odds under weighted lottery

RAG Pipeline

Files: - backend/app/services/rag_service.py for RAG orchestration -
backend/app/rag/ingest.py for document ingestion - backend/app/rag/documents/
for the source knowledge base (8 documents)

How It Works

```
graph LR
    subgraph Ingestion["1. Ingestion (one time)"]
        direction TB
        DOCS["8 text files<br/>OPT, STEM OPT, H-1B,<br/>CPT, Green Card, F-1,<br/>H-1B Wage"]
        SPLIT["RecursiveCharacterTextSplitter<br/><i>1000 chars, 200 overlap</i>"]
        CHUNKS["21+ text chunks"]
        EMBED_I["gemini-embedding-001<br/><i>768 dim vectors</i>"]
        STORE["ChromaDB<br/><i>Local persistence</i>"]
        DOCS --> SPLIT --> CHUNKS --> EMBED_I --> STORE
    end

    subgraph Retrieval["2. Retrieval (per query)"]
        direction TB
        QUERY["User question"]
        EMBED_Q["Embed query<br/><i>gemini-embedding-001</i>"]
    end
```

```

SEARCH["Cosine similarity search<br/><i>top k=4</i>"]
RESULTS["4 relevant chunks<br/>+ source metadata"]
QUERY --> EMBED_Q --> SEARCH --> RESULTS
end

subgraph Generation["3. Generation (per query)"]
direction TB
PROMPT["Assemble prompt:<br/>System Instruction +<br/>User Context +<br/>RAG Chunks"]
LLM["Gemini 2.5 Flash"]
RESPONSE["Grounded response<br/>with source references"]
PROMPT --> LLM --> RESPONSE
end

STORE -.->|"Vector store"| SEARCH
RESULTS --> PROMPT

```

Knowledge Base Documents

File	Content
opt_rules.txt	OPT eligibility, application process, deadlines, unemployment limits
stem_opt_extension.txt	STEM OPT extension rules, E-Verify, I-983, employment rules
h1b_visa.txt	H-1B cap, lottery, petition process, cap-gap, portability
h1b_wage_level_selection.txt	FY2027+ wage level weighted H-1B selection system
cpt_rules.txt	CPT eligibility, types, 12-month rule, application process
green_card_process.txt	EB categories, PERM, I-140, I-485, country backlogs, AC21
f1_general_rules.txt	F-1 status maintenance, employment options, SEVIS, violations
international_student_tax_filing.txt	Tax residency, forms, treaty benefits, FICA exemption

Technical Details

- **Splitter:** RecursiveCharacterTextSplitter (LangChain), 1000 chars, 200 overlap
- **Embedding Model:** models/gemini-embedding-001 (768 dimensions)
- **Vector Store:** ChromaDB (embedded, serverless, local persistence)
- **Search:** Cosine similarity, top k=4 results per query

AI Chat Service

File: backend/app/services/gemini_service.py

Model: gemini-2.5-flash (primary) with gemini-2.0-flash as fallback

Prompt Structure:

[System Instruction]

You are VisaPath AI, an expert immigration advisor...

- Answer questions about US immigration processes
- Provide accurate, actionable advice based on USCIS rules
- Always cite specific rules, deadlines, or requirements
- Never provide legal advice, frame as general information

[User Context]

Visa: F-1, Degree: Master's, STEM: Yes, Country: India

[Reference Documents]

<RAG retrieved chunks with source metadata>

[User Question]

Can I work for two employers on STEM OPT?

Data Layer

Immigration Rules (immigration_rules.py)

Hardcoded USCIS rules as Python dictionaries:

```
OPT_RULES = {
    "apply_before_graduation_days": 90,
    "apply_after_graduation_days": 60,
    "duration_months": 12,
    "unemployment_limit_days": 90,
}

STEM_OPT_RULES = {
    "extension_months": 24,
    "unemployment_limit_days": 150,
    "requires_e_verify": True,
}

H1B_RULES = {
    "regular_cap": 65000,
    "masters_cap": 20000,
    "registration_month": 3, # March
    "start_date_month": 10, # October 1
    "cap_exempt_employers": ["universities", "nonprofit_research", "government_research"],
```

```

}

FILING_FEES = {
    "I-765_OPT": 580,
    "I-129_H1B": 780,
    "H1B_registration": 215,
    "premium_processing": 2805,
    ...
}

```

Country Backlogs (country_backlogs.py)

Country	EB-1	EB-2	EB-3
India	2-4 years	10-30+ years	10-25+ years
China	1-3 years	4-8 years	4-8 years
Rest of World	0-1 years	0-2 years	0-2 years

STEM CIP Codes (stem_cip_codes.py)

60+ common STEM Designated Degree Program CIP codes including Computer Science (11.0701), Engineering (14.xxxx), Mathematics (27.xxxx), Data Science (30.3101), etc.

Authentication

- **Registration:** Email + password, bcrypt hashing (salt rounds: 12)
- **Login:** Returns JWT token (24h expiry)
- **Token validation:** `dependencies.py` extracts user from JWT on protected routes
- **Route guards (frontend):** `RequireAuth` redirects to `/login`, `RequireOnboarded` redirects to `/onboarding`, `RequireAdmin` restricts `/admin`

Rate Limiting

Four layers:

1. **Per-IP rate limiting** (`rate_limit.py`) protects auth endpoints from brute force
2. **Daily AI request tracker** (`ai_rate_limit.py`) limits total Gemini API calls per day
3. **Frontend pre-check** via `GET /api/rate-limit-status` lets the frontend fast fail before expensive AI calls
4. **Gemini 429 sticky marker** triggers a 5-minute cooldown after receiving a Gemini rate limit error

Frontend Architecture

State Management

`AuthContext.tsx` manages all global state: - User authentication (login, register, logout) - User profile and onboarding data - Cached timeline and tax guide - AI generation loading states (the `generating` flag drives the loading skeleton)

Routing

<code>/login</code>	> <code>LoginPage</code> (public)
<code>/onboarding</code>	> <code>OnboardingPage</code> (auth required)
<code>/timeline</code>	> <code>TimelinePage</code> (auth + onboarded)
<code>/alerts</code>	> <code>AlertsPage</code> (auth + onboarded)
<code>/actions</code>	> <code>ActionsPage</code> (auth + onboarded)
<code>/chat</code>	> <code>ChatPage</code> (auth + onboarded)
<code>/documents</code>	> <code>DocumentsPage</code> (auth + onboarded)
<code>/tax-guide</code>	> <code>TaxGuidePage</code> (auth + onboarded)
<code>/profile</code>	> <code>ProfilePage</code> (auth + onboarded)
<code>/admin</code>	> <code>AdminPage</code> (admin only)
<code>*</code>	> Redirect to <code>/timeline</code>

Key Design Decisions

- **Fire and forget navigation:** Onboarding submits data and immediately navigates to `/timeline`, showing the loading skeleton while AI generates in the background
- **Client side date recalculation:** `TimelineDashboard` recalculates `is_past` and `urgency` from the browser date via `useMemo` on every render, so cached timelines never go stale
- **Browser local date:** Frontend sends `user_today` with timeline requests for timezone correct date handling